Arquitectura de Computadoras

Jaime Nazar Anchorena

Contents

Unidad 1: Codigo en bajo nivel **3**
Programas y binarios . . . 3
Compilacino y linkedicion de C(de PI) . . . 3
Ejecucion del programa . . . 3
Representacion en memoria . . . 3
Programa en disco . . . 3
Analisis de binario(linux) . . . 3
`file` . . . 3
`strings` . . . 3
Bless . . . 5
Llamadas a funcion(call) . . . 5
Llamadas al sistema operativo(System Calls) . . . 5
Formas de ejecutar System Calls . . . 7
Procesadores y Lenguaje ASM(en Intel) . . . 7
Registros de Intel para programas . . . 7
Uso del dissasembler . . . 7
Assembler de Intel . . . 7
Instrucciones . . . 10
Modos de direccionamiento . . . 10
Registros de flags . . . 10
Ejemplo . . . 10
Compilacion y ejecucion . . . 12
Argumento por linea de comando . . . 12
Syscalls . . . 13
Assembler y C . . . 13
Instruccion `RET` . . . 13
Pasaje de parametros . . . 13
Llamado de funciones . . . 14
Convenciones en C . . . 14
Llamado de C a ASM . . . 14
Compilacion . . . 15
Deteccion stack smashing . . . 15
Secciones del programa . . . 15
64-bits . . . 15

Unidad 2: Componentes de hardware **17**
Sistema de entrada y salida . . . 17
CPU . . . 17
Mapa de memoria . . . 17
Clasificacion de memoria . . . 18
ROM(Read Only Memory) . . . 18
RAM(Random Access Memory) . . . 19
Decodificacion . . . 19
Compuertas . . . 19
Decodificadores . . . 21
Pin IO/M . . . 22
Palabra y procesador trasparente . . . 22
Mapeo en memoria . . . 23
Interrupciones . . . 23
Interrupciones de hardware . . . 23
PIC(Controlador programable de interrupciones) . . . 24
IDT(Interrupt descriptor table) . . . 24
Servicios de BIOS . . . 24
Timer tick . . . 27

Unidad 3: Gestion de memoria **27**
Modo protegido . . . 27
Conmutacion de tareas . . . 27
Proteccion de tareas . . . 27
MMU . . . 27
Descriptores . . . 29
`SEGMENT BASE` . . . 30
`SEGMENT LIMIT` . . . 30
Stack State Segment(TSS) . . . 30
Privilegios de E/S . . . 30
Paginacion . . . 30

Unidad 1: Codigo en bajo nivel

Programas y binarios

Compilacino y linkedicion de C(de PI)

Para un lenguaje se convierta en ejecutable pasa por una serie de pasos, en uno de los pasos el `.c` se conviernte en un `.s` (Assembler) y ese es el que nos interesa. `gcc` primero invoca el proprocesador `cpp`, el cual toma el coodigo fuente original y realiza la macroexpansion de diretivas como `#include` y `#define`.

Luego `gcc` toma el control de esa salida y convierte el código en C en código equivalente en Assembler. Genera un archivo con extensión `.S`. Este código en ASM puede ser generado en sintaxis AT&T ó en la sintaxis Intel. Luego el compilador GNU de Assembler "gas" se encarga de generar el codigo objecto `.o`. En Linux el ejecutable podrá ser de diferentes formatos, por ej, ELF (mas reciente) y `A.OUT` .Se debe tener en cuenta en este punto que el linkeditor `ld` también puede generar el modelo flat, también llamado binario.

Programa `.c` -> Preprocesador(cpp)(`.c`)) -> gcc (`.s`) -> gac `.o` -> Id Ejecutable

Ejecucion del programa

Linea de comando -> Sistema Operativo -> Lectura del disco -> Asignacion de espacio en memoria -> `CALL` o `JMP` a direccion de memoria de programa -> Ejecucion del programa

Representacion en memoria

Los programas antes de ser ejeutados van del disco a la memoria, esto permite ejecutarlos mas de una vez sin tener que volver a cargarlos y hace que la lectura y ejecucion del programa sea mas rapida.

- **Codigo**: Instrucciones del programa
- **Datos**: Variables estaticas y globales que si inicial al cargar el programa
- **Heap**: Memoria dinamica que se reseva y se libera en tiempo de ejecucion
- **Pila**: Argumentos y variables locales

Para que corra el programa, el sistema operativo debe dejar punteros a las distintas secciones del programa, mas especificamente: un puntero al codigo, punteros a los datos, y puntero a la pila.

Programa en disco

La primer parte se conoce homo `header` o encabezado, tiene informacion para el sistema operativo, por lo tanto, depende del sistema operativo. La segunda parte es el codigo en si.

Analisis de binario(linux)

`file`

Se puede ver que tipo de archivos es con el comando `file`, podria ser un `ELF`(Executable Linux Format)

`strings`

Es un programa que permite extraer todos los `string` que se encuentran dentro del executable en la seccion de datos. Sin embargo, no lo puedo modificar directamente, para ello conviene usar un editor hexadecimal.

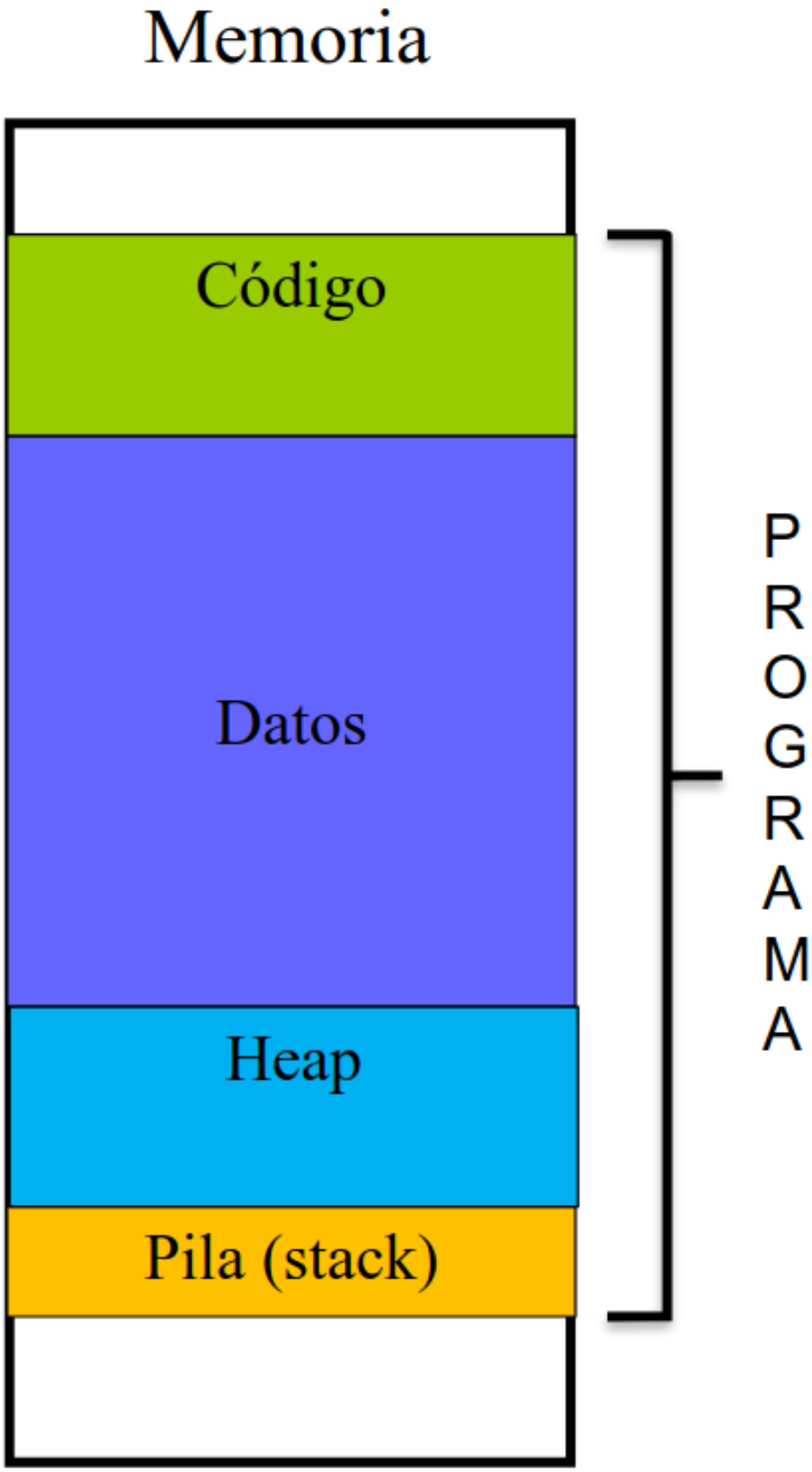


Figure 1: Representacion de un programa en memoria

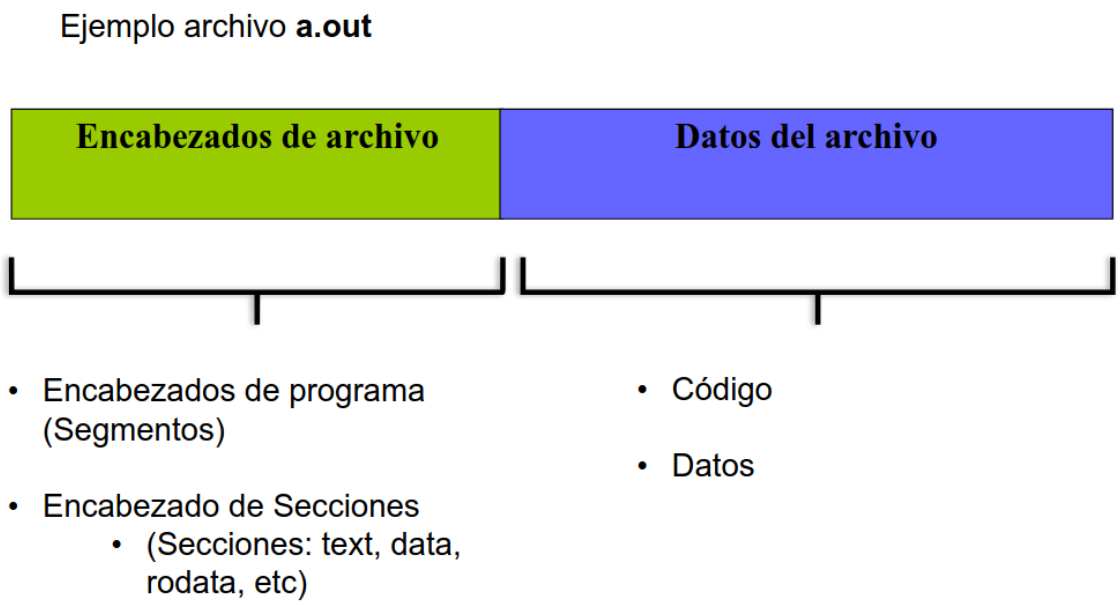


Figure 2: Representacion programa en memoria

Bless

Permite leer y editar archivos en su forma mas basica, hexadecimal(binario).

Llamadas a funcion(call)

La isntruccion `CALL` de Intel permite el llamado a una funcion, guarda en la pila la direccion de retorno. La funcion debe terminar con la instrucion `RET` de Assembler, la cual saca el puntero a la promixa intruccion del stack y el programa continua desde ahi.

```
main:
...
CALL func 1
...
func1:
...
RET
```

Toda llamada a funcion se resuelve con un `CALL`

Llamadas al sistema operativo(System Calls)

El sistema operativo se encarga de asignar memoria, verifica quien tiene permiso y quien no, dependiendo de los permisos de cada programa, estos podram hacer determinadas llamadas al sistema operativo.

User space (system call)-> Kernel space

En la documentacion, estan las declaraciones de System Calls, donde se especifica los parametros y que hace

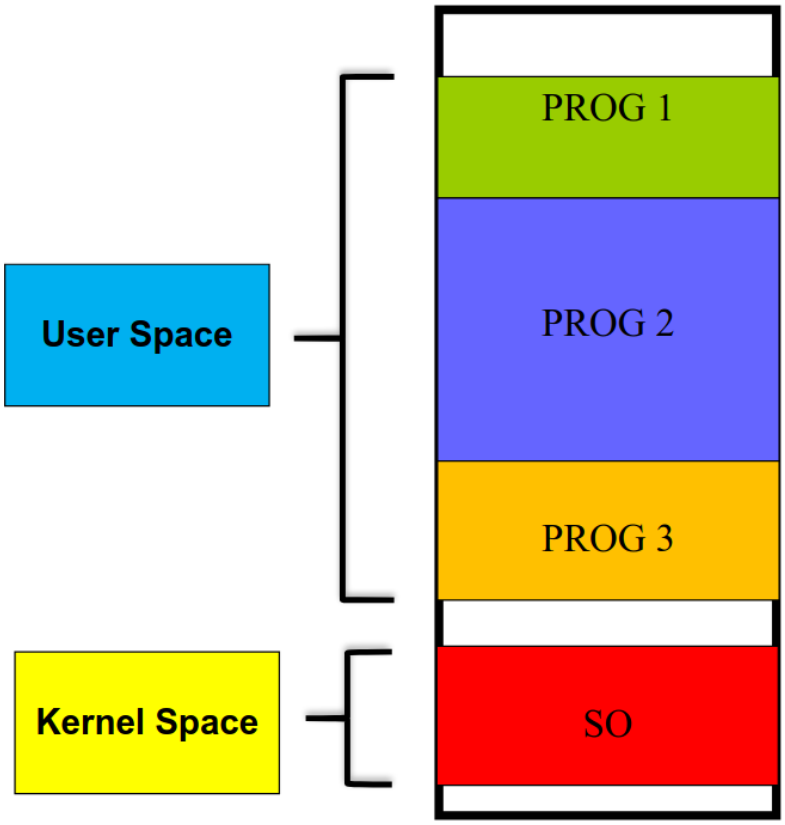


Figure 3: Representacion de memorio del sistema operativo

cada una. Estas instrucciones reciben los parametros a traves de los registros(siempre por los registros).

Ejemplo: Para imprimir en pantalla, el system call que utiliza linux es `write(1, "wawa", 10wawa)`

Formas de ejecutar System Calls

- `INT 80h`: Interrupcion numero 80, consume mas tiempo
- Instrucciones `SYSCALL/SYSRET` (Intel) y `SYSenter/SYSEXIT` (AMD): Disponibles desde Pentium II, la libreria de C depende de la arquitectura. Sirve para saber donde estoy en relacion al programa.
- `vsycall` y `VDO`: Syscall virtuales y Virtual Dynamic Shared Object, linux crea paginas de moria en user space para acelerar tiempos.

Procesadores y Lenguaje ASM(en Intel)

Registros de Intel para programas

- `IP`: Instruction Pointer(`EIP` en 32 bits y `RIP` en 64 bits)
- `SP`: Stack Pointer(`ESP` en 32 bits y `RSP` en 64 bits), apunta al final de la fila.
- Registros de manejo de memoria:
  - `CS`: Code Segment, apunta al inicio del segmento en memoria(posicion absoluta)
  - `DS`: Data Segment(Tambien `ES`, `FS` y `GS`), la direccion de la informacion desde el inicio del data(posicion relativa). Para obtener la relacion absoluta, se le suma el `CS` al `DS`.
  - `SS`: Stack Segment, apunta al inicio de la fila.

  Nota: Los registros de segmento mantienen su ttamano en arquitectura de 32 y 64 bits.

Uso del dissasembler

Basicamente tenemos que pedirle que lo decompile en el Assembler de Intel pues ese es el que vamos a utilizar en la materia.

```
objdump -d -M intel [nombre del programa]
```

> **Nota**: Siempre se puede usar `man` para ver que hace cada cosa.

Assembler de Intel

El Intel 80386 trajo un cambio de paradigma introdujendo varias features;

- Multitarea
- Multiusuario
- Tiempo compartido
- Tiempo real
- Sistema de proteccion

Lo importante de todo esto es que son **features** de hardware, el procesador las tendra independientemente del software que esta corriendo. Este procesador tendra registros de 32 bits y sosportte de hasta 4Gb de RAM.

Intel mantubo la retrocompatibilidad, con 8086 por ejemplo, manteniendo los registros `AH` y `AL`, que forman parte del `AX`(De 16 bits), el cual a su vez es mitad del `EAX`.

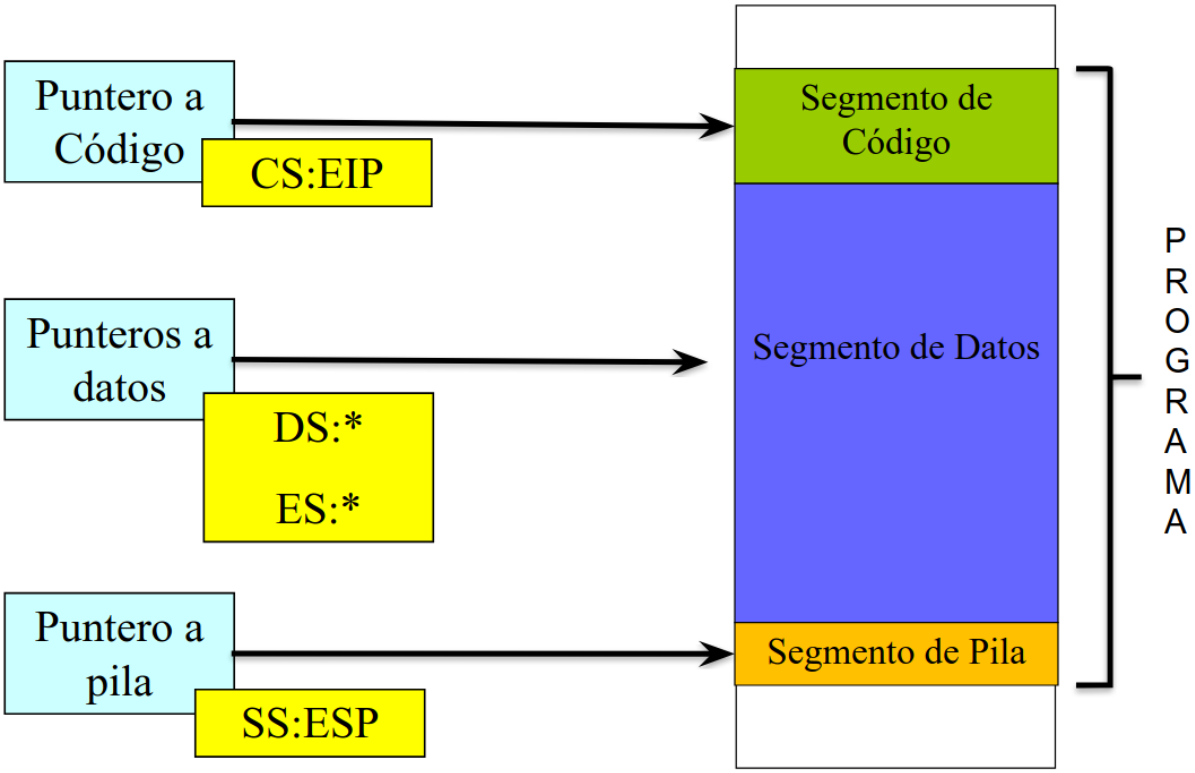


Figure 4: Representacoin de los registros en la ejecucion de un programa

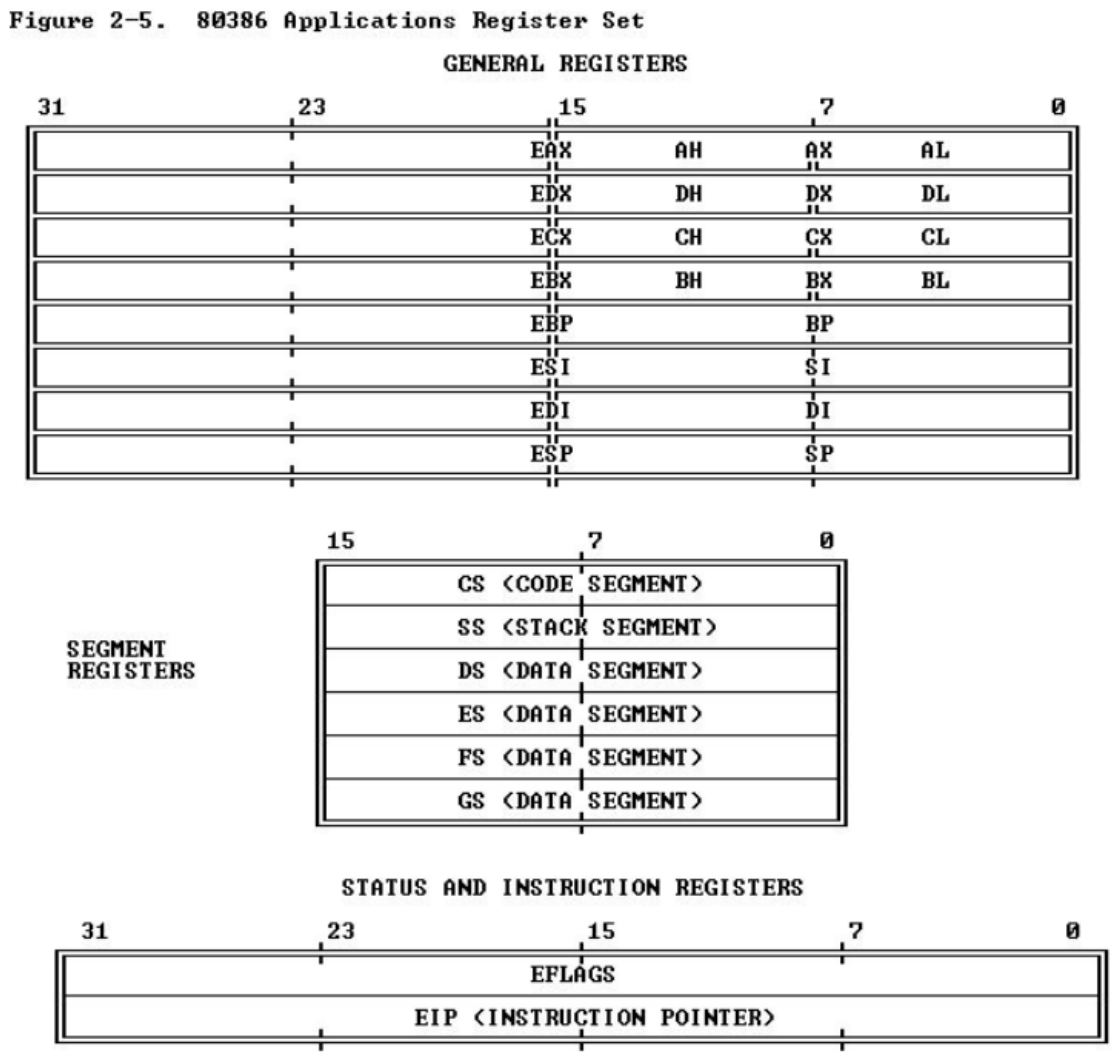


Figure 5: Registros del Intel 80886

Hay dos tipos de Assembler principales, el de AT&T y el de Intel, nosotros vamos a usar esclusivamente el de Intel.

Instrucciones

Ordenes que se le dan al procesador, por ejemplo:

```
add eax, 0x1
```

Para saber que hace cada instruccion tenemos la cartilla. Las instrucciones en Intel tienen tamano variable, si mencionas un valor numerico y no poner la `h` al final, se toma como un numero decimal. Ademas, todas las direcciones de memoria tienen un byte, si pido 16 bits de `109h`, entonce va a traee `109h` y `10Ah`.

> **Nota**: Si intentas usar una instruccion invalida, da error en tiempo de compilacion. Por ejemplo, `mov AX, 000099h` da error pues los `0` los cuenta.

Orden ejecucion: Busca direccoin de memoria -> Decodifica instruccion -> Si se modifica un registro lo hace, y si tiene que modificar memoria tiene que volver a memoria.

Para saber cuando bytes tiene que buscar o cualquier otra opcion de la misma instruccion, existe el microcodigo, el cual es interno del procesador y contiene todas las opciones, por ejemplo, `movw`, `movb`, etc. Estas se pueden entender como un switch case interno del procesador.

Modos de direccionamiento

- Direccionamiento inmediato: `mov ax, 0fffh`
- Direccionamiento de registro: `mov eax, edx`
- Direccionamiento directo o absoluto: `mov ax, [0564h]`
- Direccionamiento indirecto: `mov cx, [bp]`
- Direccionamiento como indice o indexado: `mov cd, [bp+4]`

> **Nota**: NO existe `mov [bx], [ax]`, si quiero hacer eso necesito usar un registro de intermediario.

Registros de flags

Ejemplo

```
section .text
    global _start

_start:

    mov dx,0FFh
    mov bx,20h
    add dx,bx
    push dx
    push 4
    pop cx

Ciclo:
    inc bx
```

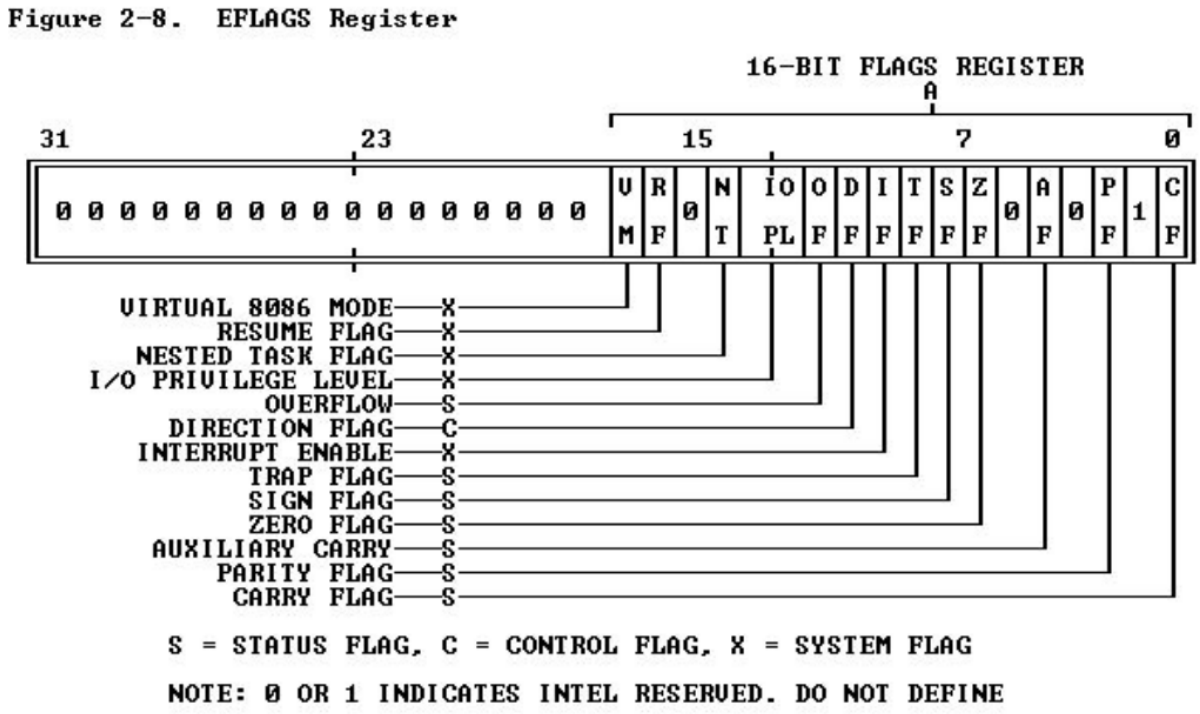


Figure 6: Registros de flags

```
    dec cx
    jnz Ciclo

    mov eax,parametros

    mov AH,[parametros]
    mov BL,[parametros+1]
    add ah,bl
    mov [salida],ah

    ret 0

section .data
parametros db 11h,12h,13h
salida db 0
```

Instrucciones como `section .text` son especificas del compilador, en nuestro caso `nasm`, y no son las mismas para todos los compiladores. Por ejemplo, el `#include<>` de C es especifico para el comilador y no lenguaje C en si.

Compilacion y ejecucion

Para compilar:

```
nasm -f elf64 ejemplo.asm -o ejemplo.o
```

Para linkeditar:

```
ld ejemplo.o -o ejemplo
```

O `gcc` tambien sirve para linkeditar. `ld` lo pasa directo a ejecutable y nada mas, mientras que `gcc` le agrega mas informacion, le agrega headers. El gcc sirve mucho para likeditar assembler con C, mientras tanto `ld` seria la opcion mas "pura".

Esto genera el ejecutabl `ejemplo`.

> **Nota**: El flag 0 lo puede levantar cualquier instruccion en cualquier momento

Argumento por linea de comando

Para destacar, el primer argumento es siempre el nombre del programa que se esta corriendo y el ultimo es `NULL`. Esto es independiente del lenguaje y es parte del funcionamiento del sistema operativo. En un sistema Unix, el sistema operativo, antes de darle control al programa configura el stack para que tenga los argumentos del programa. El Stack al inicio del programa queda:

Dir.	Data
`ESP`	Cantidad de argumentos
`ESP+4`	Path al programa
`ESP+8`	Direccion del 1er arg.
(...)	(...)

Dir.	Data
	NULL

Pero no es practico usar ESP, por ello se arma un **stack fram**, es decir, se guarda el valor de ESP en EBP.

Syscalls

Vamos a usar las syscalls de linux, sino habria que buscar en la doc cual es el codigo de lada syscall. En EAX pones el numero de la syscall que queres ejecutar y en EDX pones el numero de retorno. Hay syscall con argumentos variables y otras con argumentos tanto obligatorios como opcionales(depende del sistema operativo).

Una syscall es una funcion del sistema operativo, dependen del sistema operativo y permiten interactuar con el sistema. Estan ubicadas en el sistema operativo mismo, es decir que en tiempo de ejecucion estan en la memoria dedicada al kernel space, son parte del Kernel.

Ejemplo: Llamada a syscall print.

```
mov ebx, 1
mov eax, 4
int 80h     ; Llama a print
```

Assembler y C

Vamos a mezclar assembler con C, donde vamos a poder tener funciones y variables del lado de C, y funciones y variables del lado de assembler. Para que sean compatible entre los dos, se va a usar la pila y los registros. Desde assembler nos vamos a adecuar con lo que hace C, ya que C es fuertemente tipado y eso va a hacer la interaccion mas facil.

En un procesador Intel, la pila comienza en el tope(0x....FFFFh) y decrece a nivel puntero en la memoria, el ESP sera el **stack pointer**.

Instruccion RET

Cuando se ejecuto, el procesador toma el contenido de los apuntado por ESP y salta a esa posicion de memoria. Es equivalente a hacer un JMP a esa direccion.

Es muy importante no cambiar el esp mientras se ejecuta una funcion, o al menos dejarlo en el mismo lugar que lo econtro. Entonces es muy importante para nosotros no perder la direccion de memoria de retorno, entonces por seguridad nos cenvendria hacer una copia. Eso es lo que hace C detras nuestro, se encarga de mantener una capia y antes del return deja el stack tal y como estaba.

Si esto falla ocurre el famoso Segmentation Fault, hay que tener mucho cuidado porque en assembler no hay mucha proteccion.

Pasaje de parametros

```
int funcion(tipo1 par1, ripo2, par2) { ... }
```

El pasaje depende de la qrquitectura del compilador:

- **32 bit**: Todos se pasan por la pila. Esto genero problemas de seguridad y lo cambiaron(En memoria tambien hay codigo).
- **64 bit**: Se pasan primero por registros, y si no alcanzan(hay una cantidad finita de registros), luego se pasan por la pila.

En assembler no hay una forma determinada.

Para pasar argumentos por registro se usan `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`(Si tengo un procesador de punto flotante hay mas, pero no los vamos a ver). Si la funcion necesita mas parametros se usa la pila. Todo esto es importante para mezclar con assembler, necesitamos entender como funciona el compilador. Los registros son especificos del C, no depende del hardware.

Reglas:

- Hay registros que esta prohibido usar.
- Segun el tipo de dato se usan diferentes registros.
- Si los argumentos no entran en los registros van a la pila.

Llamado de funciones

Las funciones se llaman con la pila, esto juego un rol fundamental en arquitectura x86. En este espacio de memoria se almacenan las variables locales de la funcion llamadoa, sus argumentos y su direccoin de retorno. Aparece el concepto de **frame**, es la parte de la pila que le corresponde a cada funcion. Se crea en tiempo de ejecucion.

Convenciones en C

Los parametros se pushean en el stack de derecha a izquierda, para backupear `ESP` lo vamos a guardar en otro registro, en `EBP`(base pointer). Antes de asignarle `ESP` a el `EBP`, se pushea el `EBP` al stack para restaurarlo despues. Esto se conoce como el armado del **stack frame**.

```
push ebp
mov ebp, esp
```

Luego para desarmar el stack es la operacion opuesta.

```
mov esp, ebp
pop ebp
```

Estas son las operaciones que realiza una funcoin en C para asegurarse de no perder el `ESP`. Para retornar un valor, si el valor es menor a 32 bits se retorna el `EAX`, si es mayor retorna la parte alta en `EDX` y la parte baja en `EAX`. Si es mas complejo (ej. Estructura de datos) retorna un puntero formado por `EDX:EAX`. En el decompilado de C existe una macro llamada `leave`, esta realiza el desarmado del stack frame.

> **Nota**: Se llama base donde empieza la pila.

Llamado de C a ASM

En C utilizamos:

```
#include<stdio.h>
```

```
extern unsigned int siete(void);

main() ....
```

Notar la palabra clave `extern`, luego en al assembler que luego voy a linkeditar con la parte de C:

```
...
siete:
    ...
```

Si no armo el stack frame, no va a pasar nada, pero es mucho mas prolijo y buena practica hacer.

> **Nota**: Numca usar `__asm__("movl $ $0x12345, %eax");`, se conoce como inline asm en C(Al profesor le parece un horror).

Tenemos dos formas de ver el codigo C convertido en ASM:

- Compilar con la flag `-S`
- Usando GDB

Va a haber operaciones que podrian no tener sentido, pero para que el compilador pueda compilar la infinidad de combinaciones diferentes de programas hay veces que lo que resulve dan ese tipo de cosas. Lo que ocurre es que el compilador va de a bloques y siempre avanza, no lo vuelve a revisar.

> **Nota**: `mov DWORD PTR[esp], 0x3` especifica que queres que el numero 3 sea guardado en 4 bytes(a pesar de que ocupa 2 bits como minimo).

Compilacion

Para la mayoria de los casos la compilacion se hace en una sola pasada, eso causa que al analizar codigo compilado aparescan operaciones que pueden no tener sentido, aunque no afectan el funcionamiento del codigo. Por ejemplo, reservar espacio de mas en la pila. Sin embargo, hay casos en los que si se hacen mas de una pasado, por ejemplo, los sistemas operativos, los cuales tienen millones de lineas de codigo y necesitan toda la optimizacion que puedan conseguir.

Deteccion stack smashing

Lo que hace `gcc` cuando no agregas ninguna flag, pone en el stack lo que se conoce como `canary`. Si al ejecutarse se detecta que cambio alguno de los `canary` entonces interrumpe la ejecucion pues detecto un `stack smashing`(aborta ejecucion).

Lo pone justo despues de armar el stack frame, el valor es un valor random que saca de una seccion de memoria. Luego, antes de que termine el programa llama a una funcion llamada `__stack_chk_fail` para verificar si el `canary` siga igual.

Secciones del programa

64-bits

GCC por convencino, al recibir parametros por registros, lo primero que hace es copiar los valores al stack. Si estoy solo usando assebler puedo usar directamente los registros. No hace `push`, sino que primero hace `sub` del `rsp` y va haciendo `mov`.

Section type	Name prefix	Description
code	.text	program code
neardata	.zdata	initialized __near data
fardata	.data	initialized __far data
nearrom	.zrodata	constant __near data
farrom	.rodata	constant __far data
nearbss	.zbss	uninitialized __near data (cleared)
farbss	.bss	uninitialized __far data (cleared)
nearnoclear	.zbss	uninitialized __near data
farnoclear	.bss	uninitialized __far data
a0data	.data_a0	initialized __a0 data
a0rom	.rodata_a0	constant __a0 data
a0bss	.bss_a0	uninitialized __a0 data (cleared)
a1data	.data_a1	initialized __a1 data
a1rom	.rodata_a1	constant __a1 data
a1bss	.bss_a1	uninitialized __a1 data (cleared)
a8data	.data_a8	initialized __a8 data
a8rom	.rodata_a8	constant __a8 data
a8bss	.bss_a8	uninitialized __a8 data (cleared)
a9data	.data_a9	initialized __a9 data
a9rom	.rodata_a9	constant __a9 data
a9bss	.bss_a9	uninitialized __a9 data (cleared)

Figure 7: Secciones del programa y para que son

Unidad 2: Componentes de hardware

Vamos a analizar como se comunica el procesador con el resto del hardware. Para representar 0s y 1s en un cable de cobre hay varias maneras, una manera es voltaje o no voltaje, otra seria +3,3V o -3,3V. El procesador se comunicara de esa manera, hay dos maneras:

- **Transmision serie**: Va todo por el mismo tiempo, cada bit ocupa un paso.
- **Transmision paralela**: Si quiero transmitir 8 bits en un solo paso, es la manera mas rapida y la que se usa en la computacion, se conoce como un **bus**.

En las computadores y circuitos chicos tenemos los bus, pero en las redes que conectan al mundo y usan miles de kilometros de cable se utiliza la transmision en serie a pesar de ser mas lente(debido al costo principalmente).

Hubo dos propuestas principales para como estructurar la conexino del CPU con memomria:

- **Von Neumann**: Propone un solo bus entre la memoria(codigo y datos) y el CPU
- **Harvard**: Propone dos buses, uno a la memoria con los datos y otro para la memoria(otra memoria) con el codigo. De esa manera de paraleliza la lectura.

Sin embargo, hoy en dia se utiliza Von Neumann a pesar de ser mas lento debido al costo y complejidad reducida.

Sistema de entrada y salida

Tenemos dos buses principales, un bus de **direcciones**(address)(A0 A1 … A15) y otro de **datos**(D0 … D7). Estan todos los perifericos conectados a los mismos cables, para que no se crucen y se comuniquen todos de manera ordenada se utiliza el bus de direcciones, este se encarga de apuntar a la direccion de memoria que quiero escribir o leer. Para saber si ecribir o leer, hay una linea extra **`r/w`** que con `1` indica `read` y con un `0` indica `write`. Los primeros procesadores tenian 8 y 8 bits para cada bus, pero se fue agrandando a medida que fue aumentando el tamano de los registros, hoy en dia son de 64 bits cada uno.

Sin embargo, hay que tener cuidado, si llenamos el bus de direcciones con direcciones de memoria, el CPU no va a poder comunicarse con otra cosa que no sea la memoria(otros perifericos, por ejemplo). Entonces, hay que ser conciente de como se utiliza el bus de memoria.

> **Nota**: Para que algo sea de 64 bits todo debe ser de 64, bus de datos, registros, etc.

CPU

Presenta la unidad de control, que se encarga de recuperar intrucciones de memoria, decodificarlas y escribir en memoria, y la unidad de ejecucion, la cual lleva a cabo la ejecucion de la intruccion. Ademas, tiene los registros y los flags. Que pasa cuando se enciende un procesador? Cual es la primer instruccion ejecutada? La primer instruccion sera la que este en la direccion que apunte el IP, pero en memoria no puede estar porque es volatil. Lo que ocurre es que primero lee instrucciones de la ROM, mejor conocido como la BIOS.

Mapa de memoria

Todas las direcciones que puede acceeder el CPU, no puede ver mas alla de lo que permite su bus de direcciones. Es decir, si tengo un bus de direcciones de 16 bits, tonces solo tengo 64K de memoria disponible.

Los CPU de Intel hacen algo diferente al expandir a 32 bits(por ejemplo), si de la nada empieza a traer 4 bytes en vez de 1 se rompe todo el codigo viejo. Por lo tanto, para mantener retrocompatibilidad el cpu busca 8 bits a pesar de que el bus es de 32 o 64 bits. Entonces, en el bus de 8 bytes, de una direccion se traen los 8 bytes, pero cada uno tiene una direccion consecutiva.

Luego, los mapas de meoria de intel siempre van a tener un byte de ancho, independientemente del bus.

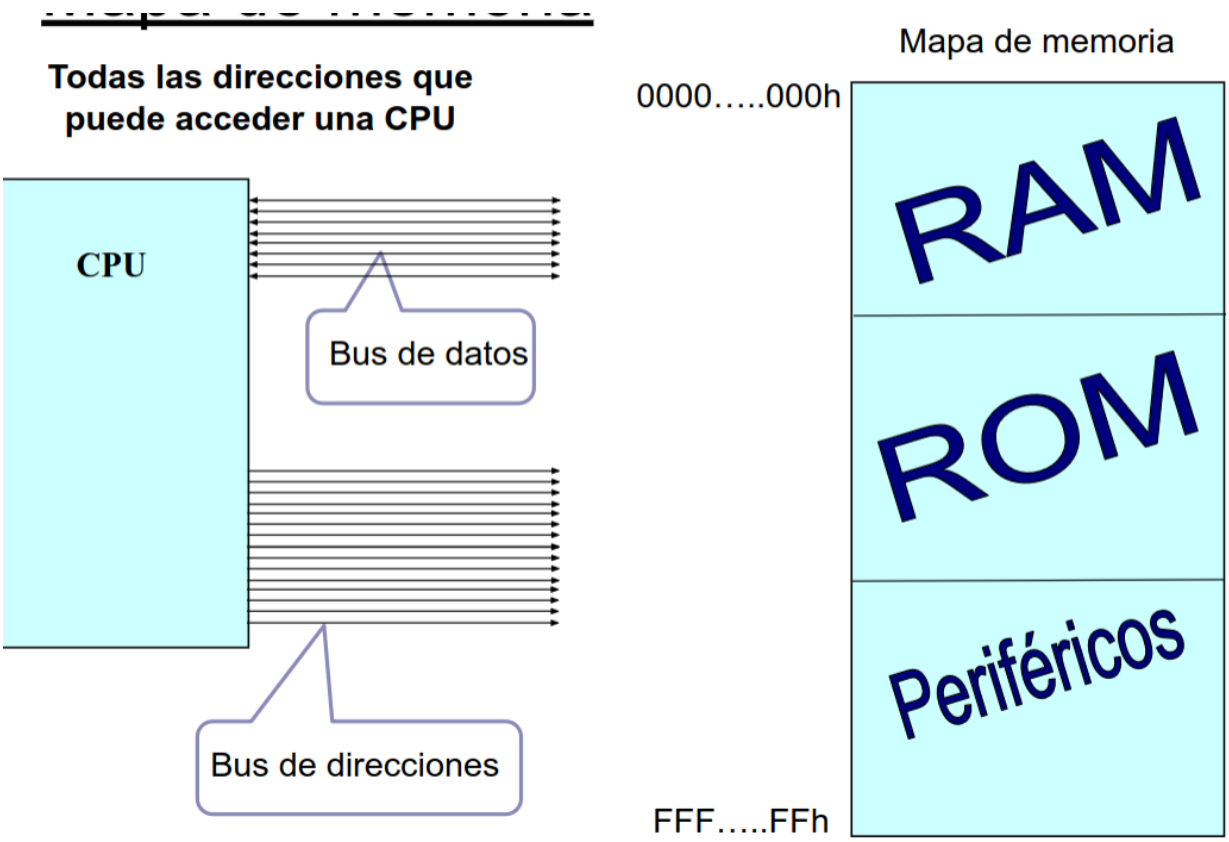


Figure 8: Mapa de memoria

Nota: K en informatica es una constante que se conoce como 2^10=1024, M es 2^20.

Clasificacion de memoria

Hay dos grandes grupos de memoria, RAM y ROMM

ROM(Read Only Memory)

Primer tipo de memoria, sean PROM, EPROM(esta se puede borrar), Flash o EEPROM. Mantinenen la informacion sin energia, no volatil, sin embargo la escritura es mas lenta que la RAM. En la PC, se encuentra la BIOS en la ROM, este sera el primer codigo que ejecutee el CPU. Sin embargo, no dura para siempre pues usa capacitores, similar a la DRAM, pero tienen una gran resistencia, por lo tanto eventualmente se va a perder. Suelen durar alrededor de 10 anios. Si lo escribis se reinicia el ciclo, pero si solo lo lees se sigue descargando.

RAM(Random Access Memory)

La mas rapida, existe DRAM(Dinamic RAM) y SRAM(Static RAM), pierde la informacion sin energia.

- **DRAM**: Necesita refresco de valores cada n milisegundos, menos compleja, mas economica y mas lenta. Por lo general es la que se tiene en las computadoras. Una **latencia** de entre 50 y 150 ns.
- **SRAM**: No necesita refresco, mas compleja, mas cara y mas rapida. Se suele utilizar para cache. Con una latencia(tiempo que tarda en devolver un dato) de menos de 10 ns.
- **SDRAM**

  Nota: A diferencia de la latencia, la **transferencia** es lo que tarda en viajar la informacion de un lugar a otro.

Decodificacion

Para conectar mas de un dispositivo al procesador, tenemos que hacer "decodificacion". Para hacer esto necesitamos herramientas electronicas:

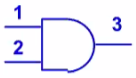
- Compuertas
- Decodificadores

Compuertas

Los circuitos integrados vienen con mas de una, se pueden usar simultaneamente pues son independientes. Sin embargo, hoy en dia se utilizan compuertas en vez de decodificadores.

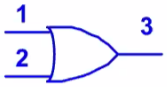
1. NOT
2. OR
3. AND
4. NAND, es como tener una compuerta AND seguida de una NOT
5. NOR
6. XOR

AND



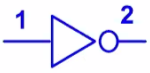
B	A	B.A
0	0	0
0	1	0
1	0	0
1	1	1

OR



B	A	B+A
0	0	0
0	1	1
1	0	1
1	1	1

NOT

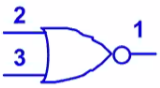


A	$\overline{A}$
0	1
1	0

NAND

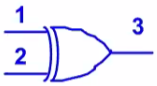
B	A	$\overline{B.A}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR



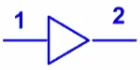
B	A	$\overline{B+A}$
0	0	1
0	1	0
1	0	0
1	1	0

XOR



B	A	$B \oplus A$
0	0	0
0	1	1
1	0	1
1	1	0

BUFFER



A	A
0	0
1	1

NOTA: El buffer es muy de bajo nivel y no lo vamos a ver, pero basicamente si tenes 4.8V lo pasa a 5V para que llegue a ser un 1.

Decodificadores

La idea es que cada periferico se encienda cuando el procesador intente de acceder a las direcciones de memoria a la que estan mapeados.

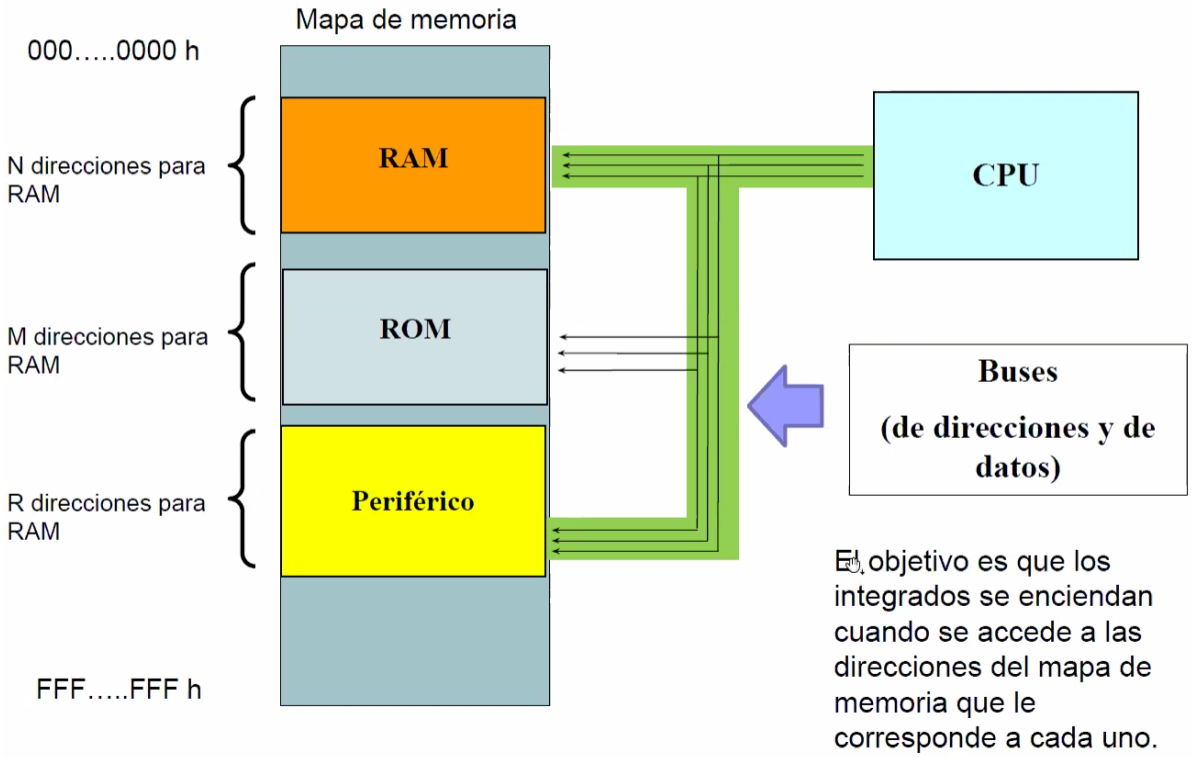


Figure 9: Buses de memorias

Del bus de direcciones van a salir cables al **circuito decodificador**, entonces cuando quiera acceder a la RAM, el circuito decodificador enviara una senal para activar la RAM, se conoce como CS(Chip Select). Entonces, cuando se cumplen las direcciones que corresponded a la RAM, se enviara un 1 por CS.

Ningun periferico esta construido para ir en una direccion de memoria, solo pueden ver su propia memoria interna. Hay dos sistemas que vamos a ver:

- **Sistema 1**: Un procesador y una ROM, la ROM ocupa toda la memoria. Entonces, deja el CS en 1 siempre.
- **Sistema 2**: Varios perifericos a la vez.

Los decodificadores tienen una tabla de verdad donde una salida siempre es 1. Entonces, por ejemplo, existe un decodificador que pasa 3 inputs a 8 outputs mediante una tabla de verdad. Se prende el output 4 si el numero en binario de los 3 inputs es 4, eso es basicamente lo que hace la tabla. Muy importante, solo una esta prendida a la vez, el resto se mantiene apagadas.

Ejercicio: Se dispone de un microprocesador con 16 lineas de bus de direcciones y 8 lineas de bus de datos. Se desea conectar el procesador con dos integrados de ROM de 32k x 8. Aclaracion: Siempre esta el bus de control aunque no se lo mencione(sino no funcionaria).

Analizando, con 16 bits de direcciones, tenemos 2^16 direcciones, entonces son 2^6 * 2^10 y eso significa que son 64K de direcciones. Como es como cada direccino son 8 bits, seran hasta 64KB de memoria.

Entonces, cuando la direccion que envie el procesador tenga el A_15 = 0, entonces el chip select de la primer ROM sera 1. Caso contrario, el chip select de la otra ROM sera 1. Notar que generalmente el circuito codificador se termina de definir al final del analisis.

> **NOTA**: La suma de todas las patitas que van al dispositivo y las que van al decodificador deben ser igual a la suma de patitas que tiene el bus de address. Sirve para verificar que hayas resuelto bien los ejercicios.

Los decodificadoes se pueden reutilizar solo si el tamano de los bit significativos siguen siendo los mismo, sino va a ver que usar otro. (Santi recomienda ver dark mirror).

Ejercicio: Si tengo un sensor de humedad y temperatura, este tendra una patita read, otra CS, 8 para el BD y una para la direccion(o temp o humedad). Por lo tanto necesito un decodificador que tome todas las direcciones y para una direccion prenda el CS del sensor.

> **NOTA**: Si solo podes usar un deco con 3 patitas y solo vas a usar una, conectar las patitas restantes a la misma. Dejarlas sin conectar las deja vunerables a ruido. **NOTA**: La ROM generalmente conviene colocarla al final del las direcciones de memoria debido a que es muy poco probable que se modifique, mientras que la RAM puede aumentarse, etc. De manera que deja espacio para perifericos que sean modificables.

Pin IO/M

En los primeros procesadores, Intel necesitaba poder acceder a mas perifericos sin tener que modificar los registros. Entonces, se agrego al CPU un pin llamado IO/M(Input/Output memory(en realidad la barra deberia ir arriba de la M para indigar que esta negada)), de manera que permitio tener dos mapas de memoria, uno para la memoria y otro para los perifericos(basicamente son identicos). Para ello se crearon dos intrucciones nuevas de 16 bits, `IN` y `OUT`. Si un programadoe escribe `mov ah, [1234h]`, la direccion va a salir por BA, el IO/M va a estar en 0, read en 1 y write en 0. IO/M va a salir al decodificador para prender el CS de la memoria. Si un programador escribe `IN ah, 1234h`, notar que **no** tiene corchetes, el IO/M va en 1 y el CS de algun periferico se va a prender. Basicamente, son maneras de agregar funcionalidad, como `*` en C.

> **NOTA**: Nosotros solo vamos a usar cosas que funcionan con PC, por ejemplo, el teclado esta en la direccion 60h.

Palabra y procesador trasparente

Una palabra para un procesador generico siempre sera la cantidad de bits del BUS de datos o de los registros(La cantidad de bits que se pueden obtener en un ciclo). Por otro lado, en ASM Intel, una `word` son 2 bytes. Un procesador transparente es aquel que realiza su función sin alterar el contenido original o sin que el usuario lo perciba directamente. Por ejemplo, las memorias de 8 bits deben funcionar con el procesador de BD de 16 bits.

Mapeo en memoria

Un dispositivo es manejado como una o varias posiciones de memoria. Cada posicion de memoria puede ser un registro interno diferente del dispositivo. Por retrocompatibilidad, incluso despues de la introduccion del mapa IO, hay perifericos que quedaron del lado de memoria, como la pantalla en modo texto. Luego, mapear perifericos en memoria es basicamente poner perifericos en el mapa de mamoria, esto tiene ventajas y desventajas:

- **Ventajas**: Se utilizan las instrucciones de acceso a memoria, por lo tanto las alternativas de programacion son mayores por tener mayor cantidad de instrucciones para manejo de memoria. Ademas, se modifica directamente los registros del periferico sin necesidad de obtener el valor con las instrucciones IN y OUT.
- **Desventaja**: Reduce cantidad de memoria. El impacto es minimo relacionado con la cantidad de memoria que tienen las PC actuales.

Es decir, una razon para poner los perifericos en el mapa de memoria es que hay mas instrucciones que soportan `[]`, mientras que solo hay dos que soportan el mapa IO.

Para escribir en pantalla en modo texto, el mapeo en memoria de la tageeta de video sera a partir de **B8000h**, esto es asi para todas las computadoras con arquitectura x86.

Interrupciones

Hay dos maneras de comunicarse con un dispositivo:

- **Polling**(Sondeo): Pregunta constantemente si tiene un dato nuevo. Usa mucho recursos, pues esta todo el tiempo preguntando. Es muy rapido.
- **Interrupciones:** Esperar a que el periferico avise que tenga un dato nuevo. Tardan en resolverse, pero no le saca recursos al procesador.

Nosotros nos vamos a enfocar en interrupciones, polling es simplemente un loop.

Una interrupcion es una senal externa que interrumpe al micro para requerir un servicio de atencion. Una vez recibe la senal, detiene el programa que esta corriendo y ejeccuta el programa de atencion de interrupcion, luego sigue.

La interrupcion puede ser generada por un evento externo(INTR o NMI) o por la instruccion INT. Una vez termina la rutina de interrupcion, termina con `IRET` y no con `RET`, esto basicamente vuelve al programa original. Esto hay que usarlo con cada rutina de interrupcion y la rutina se debe encargar de dejar todo como estaba antes de la interrupcion. Hay distintos tipos de interrupciones:

- **Interrupciones de hardware**: Se interrumpe la ejecucion del programa activando alguna de las dos entradas que tiene el procesador(INTR y NMI). Es un evento externo al procesador.
- **Interrupciones de software**: Se interrumpe la ejecucion del programa al ejecutar la instruccion de assembler INT. Por ejemplo, `INT 44h`, donde 55h es el numero de rutina de interrupcion a ejecutar.

Interrupciones de hardware

El flag **IF** indica si se debe atender a las interrupciones externas. Si IF=1, esta habilitado, si IF=0, esta deshabilitado. Dependiendo de la patita, la interrupcion va a it por:

- **Interrupcion enmascarable**: Ingresan por INTR, el flag IF se controla con las intrucciones `sti`(set interrupts, IF=1) y `cli`(clear interrumps, IF=0). Aca van las no critica, mouse, teclados…
- **Interrupcion no enmascarable**: Las interrupciones que ingresan por la patita NMI no pueden ser enmascaradas. Y siempre ejecutan la rutina que se encuentra en la posicion 2h del vector de interrupciones.(`INT 2h`). Aca van las criticas, como el sensor de temperatura, el boton de encendido.

PIC(Controlador programable de interrupciones)

Como no alcanzaban las patitas de interrupcion para todo los perifericos, ss agrego otro periferico llamado **PIC**. Recibe ocho patitas(llamadas **IRQ**, interrupt request), las gestiona y decide cual pasa al procesador, cada una con su propia mascara. Hay que programarlo, hay que decirle cual pin tiene maas prioridad que otro y se puede controlar la mascara de cada uno individualmente. El PIC trabaja en las direcciones de E/S 20h y 21h. Ademas, luego de terminar con un interrupt, hay que mandar la senal **EOI** (End Of Interruption) al PIC, sino el PIC no va a mandar mas interrupciones.El CS del PIC se activa con 0, como se ve en el diagrama, tiene el circulo de negado. El valor que hay que enviar para senalizar el EOI, hay que mandar 20h a la direccion 20h(`OUT 20h, 20h`).

Ademas, se pueden agregar PIC en cascada, teniendo el master como el principal y el slave como secundario, el slaave interrumpe al master y luego el master interrumpe al procesador. El INTA se manda a los dos, ya que el procesador no sabe cual fue el que llamo la interrupcion. Se pueden agragar mas que dos PICs, hoy en dia se pueden encontrar hasta 15 PICs en cascada, y cabe aclarar que el segundo PIC no va a estar en 20h y 21h, sino que va a estar en otra direccion de memoria.

IMR Los puertos de entrada y salida en la PC son el 20h y el 21h, el 20h se utiliza para programar el PIC(lo utiliza el BIOS para arrancar el sistema). En el puerto 21h podemos acceder al registro IMR(Innterrupt Mask Register) del 8259, donde podemos setear que interrupciones llegan al microprocesador.

> **NOTA**: Arrancar TP con todos los IRQ en 1(todos enmascarados) para que no falle. El teclado va al IRQ1

IDT(Interrupt descriptor table)

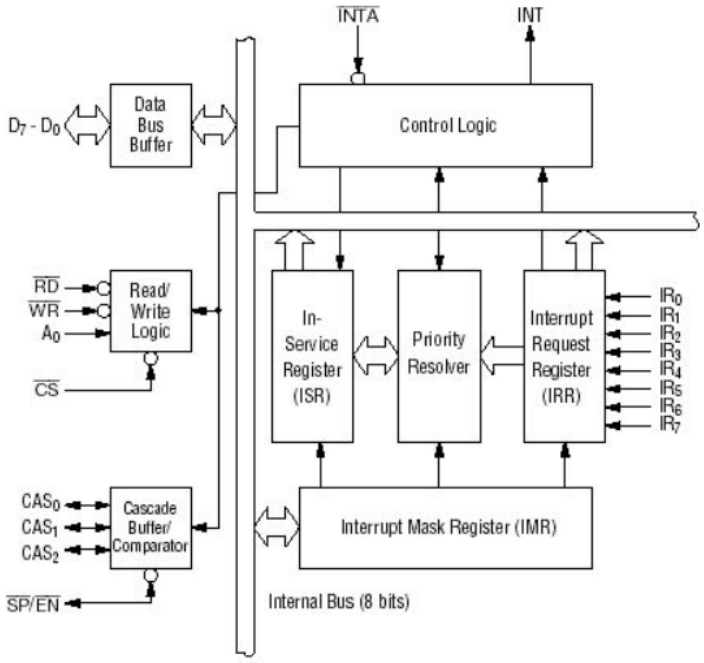
Por cada interrpupcion, tiene una tabla con el puntero a la rutina correspondiente a cada interrupcion. Hay otra patita que sale del procesador llamada **INTA**(Interrupt acknowledge), funciona de manera de handshake. Una vez recibio el interrupt, el procesador le indica al PIC con la senal de INTA que esta listo para recibir informacion, luego el PIC manda por el bus de datos la informacion del periferico.

Servicios de BIOS

El BIOS al iniciar la PC guarda en memoria rutinas basicas para poder empezar a operar. Relacionado con interrupciones de software, hay varias rutinas utiles suministradas por la BIOS y estan estandarizadas. Por ejemplo:

- `INT 10h`, rutinas de video
- `INT 13h`, rutinas de disco
- `INT 14h`, rutinas para puerto Serie
- `INT 19h`, rutinas para bootloader
- `INT 1Ah`, rutinas para el RTC

PIC – Diagrama lógico



MSM82C59A-2 Internal Block Diagram

Figure 10: Diagrama del PIC

Línea IRQ	INT Tipo	Descripción
IRQ0	08h	Timmer tick (18,2 veces por seg.)
IRQ1	09h	Teclado
IRQ2	0Ah	INT desde 8259A esclavo
IRQ8	70h	Servicio de reloj en tiempo real.
IRQ9	71h	Redireccionamiento por soft. a IRQ2
IRQ10	72h	Reservada
IRQ11	73h	Reservada
IRQ12	74h	Reservada.
IRQ13	75h	Coprocesador numérico.
IRQ14	76h	Controlador de disco rígido.
IRQ15	77h	Reservada.
IRQ3	0Bh	COM2
IRQ4	0Ch	COM1
IRQ5	0Dh	LPT2
IRQ6	0Eh	FLOPPY
IRQ7	0Fh	LPT1

Figure 11: Interrupciones de hardware por default

Timer tick

Por default, cada 55ms milisegundos genera una interrupcion, mapeador a IRQ0 con una interrupcion de tipo 08h(Para ver mas mapeos ver tabla). Esto se debe a que anteriormente, las computadoras, teniendo un procesador, corrian un programa (o lote)hasta que terminara y recien ahi pasaba al segundo. Por lo tanto, el timer tick interrupe los procesos cada 55ms y luego se va a correr una rutina que va a decidir quien es el proximo programa en ser ejecutado, esta rutina se conoce como **scheduler**. El flag de timer tick, IRQ0 esta protegido y no puede ser cambiado por cualquier programa. Estos sistemas donde se deben interrumpir los programas ya que ningun programa va a ceder el IP para dejar correr a otro, esto se conoce como sistemas preemptivos. Por lo tanto, esto es lo que da la sensacion de que todos los programas estan corriendo al mismo tiempo.

Unidad 3: Gestion de memoria

Modo protegido

Conmutacion de tareas

Lo mencionado del timer-tick, habilita correr varios programas casi en simultaneo debido a la velocidad a la cual ocurre todo, basicamente va intercambiando el programa que esta corrienda cada cierto intervalo de tiempo. El **scheduler** es una rutina del sistema operativo que decide que tarea/programa sera el siguiente en ser ejecutado luego de la interrupcion. El scheduler manda el `IP` al codigo de la siguiente tarea, esto trae un problema, necesito saber cual es el momento en que se interrumpio la tarea anterior. Esto se soluciona guardando el **contexto**(todos los registros) en memoria para tener listo todo para continuar cno la ejecucion mas tarde.

> **Nota**: El SO es una tarea, tiene la ventaja de ser la primera que corre, entonces es la que gestiona.

Que pasa si no hay tareas? Corre una tarea "tonta" que es basicamente un loop que no hace nada, en Windows es el proceso 0 llamado **Proceso inactivo del sistema**. Cuando no tiene a quien entregarle el procesador, se lo da a este programa.

Proteccion de tareas

Esto exite para evitar que los programas malignos hagan cosas como leer el espacio de memoria de otras tareas, de esa manera podrian robar informacion. El unico que se da cuenta que hay un programa accediendo a una zona de memoria que no es de el es el hardware, pues el SO u otros no se estan ejecutando. Entonces, el SO, crea unas tablas que indica que programas pueden acceder a que secciones de memoria, estas tablas son chequeadas por el CPU antes de permitir que un programa lea/escriba en memoria. Este sistema fue creado por Intel.

MMU

Es lo que valida cada acceso a memoria y determina si esta bien o esta mal a partir de las tablas. Se le va asignando espacio a los programas, este espacio puede ser random(lo que pida coada programa) o fijo. Entonces, una vez termina uno, ese espacio termina siendo liberado(desasignado), y puede haber programas que necesiten la misma cantidad de memoria que se libero antes o requieran mas y no entren. La **Unidad de Segmentacion** se encarga de asignar el espacio variable, mientras que la **Unidad de Paginacion** asigna

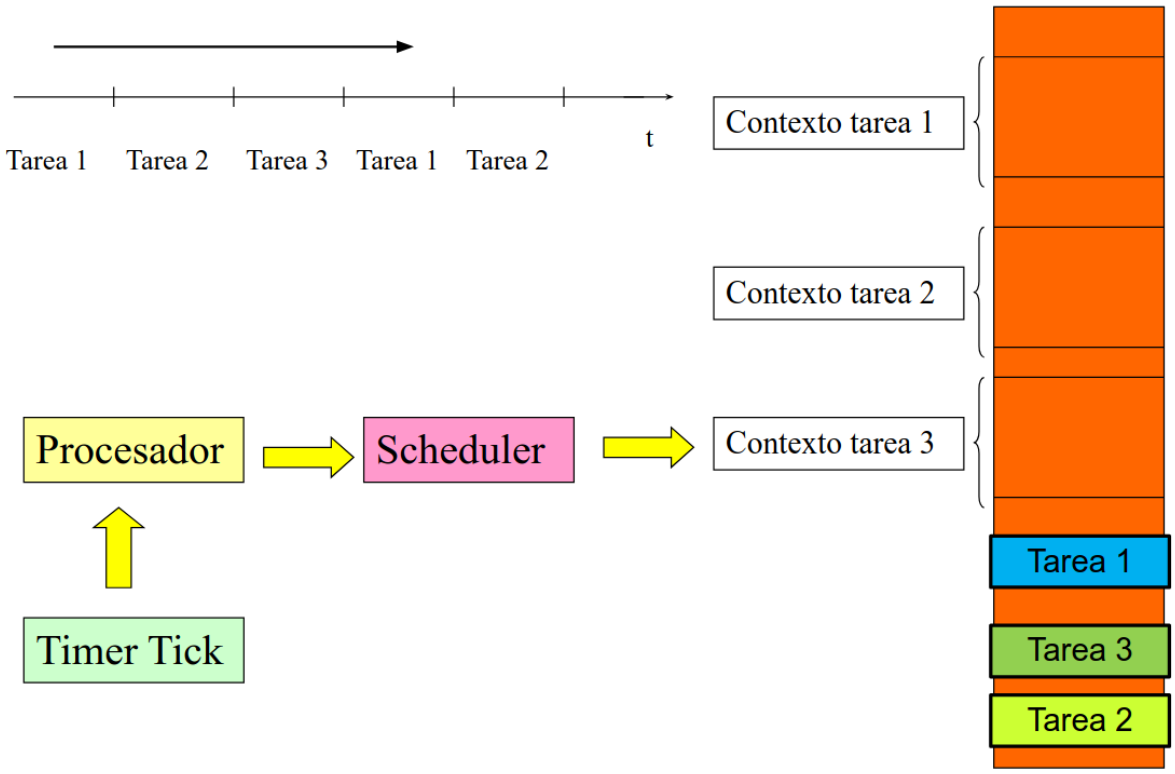


Figure 12: Diagrama scheduler

paginas de memoria(y es la que chequea las tablas), este fue el sistema que implemento Intel. La unidad de paginacion puede ser desactivada, pero la de segmentacion no(pero se puede armar toda la memoria como un gran segmento y se skipea la unidad de segmentacion, es una manera de 'desactivarla').

- **Tamano variable**(Unidad de Segmentacion): Permite dar el espacio perfecto, pero una vez termina el programa queda un espacio vacio y hay muy pocas probabilidades de que otro programa requiera exactamente el mismo espacio(poco facil de sustituir).
- **Tamano fijo**(Unidad de Paginacion): Se da memoria de a bloques, en el peor de los casos sobra espacio. Las sustituciones son mucho mas simples.

  Nota: En el TPE vamos a usar un gran segmento de memoria que va a ocupar toda la memoria, para desabilitarla virtualmente.

Unidad de segmentacion Segun Intel, son dos tablas **GDT**(Gocal Descriptor Table) y **LDT**(Local Descriptor table), dentro de la GDT van todos los descriptores de segmentos del SO y desntro de la LDT van los descriptores de todos los segmentos del resto de las aplicaciones. Se encuentran en memoria, el procesador tiene dos registros, `GDTR` y `LDTR`, que apuntan a estas tablas. En la actualidad nadie usa la LDT, solo se trabaja con la GDT. Estas tablas son las que deja el SO antes de ser interrumpido para que corra otro programa.

Sin embargo, que impide que un programa modifique la tabla de descriptores? Hay un descriptor llamado **descriptor de GDT** que se describe a si mismo, se pone los permisos mas altos dentro del procesador y el unico programa que puede modificarlo es el SO(Kernel Space, segmento de memoria con los permisos mas

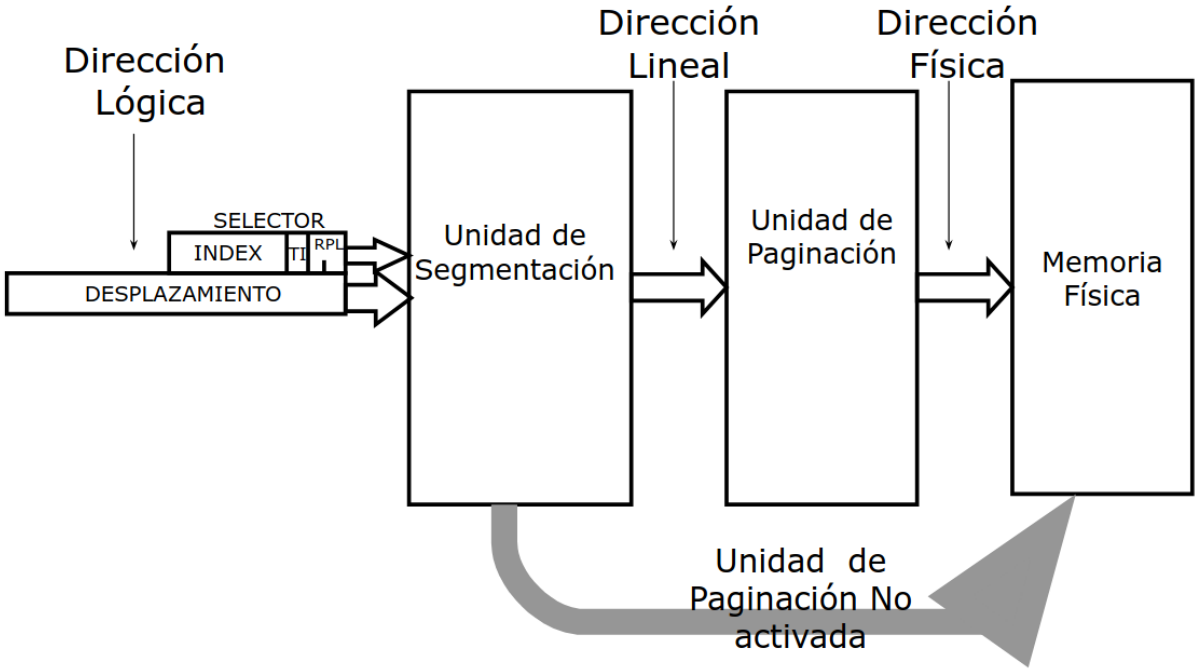


Figure 13: MMU

altos) pues, al ser el primero que llega, se pone el permiso mas alto a el mismo y al resto de las aplicaciones se les asigna un permiso menor. De esa manera, se evita que otro programa que no sea el SO modifique las tablas.

Ejemplo: Si corriendo el programa con `IP`=10xxh, descriptor dice que el programa en esas direcciones tiene entre 1001h y 1FFFh, si quiere acceder a 3222h al verificar con GDT saltara un error.

Ejemplo de uso: El code segment trabaja con el instruction pointer, el CS apunta al inicio del codigo y el IP al offset. Luego el CS no apunta exactamente a memoria sino que saca un descriptor del GDT(base apunta al inicio de segmento), y de ahi saca en inicio del segmento. Es decir, si CS es 20h, al leer la memoria va a la GDT y la GDT le devuelve la lectura de la direccion equivalente especificada en el descriptor, se conoce como **indireccion**. Entonces, puede ser que sea la direccion 20h de memoria real o ABBA000h de memoria real.

Esto se usa para organizar mejor y para controlar los permisos, agrega indireccion(o mapeo) y seguridad. Luego, si no hay espacio suficiente, se usa el `P`(persistencia), y de esa manera si no hay memoria, le da al programa espacio en disco. Las aplicaciones estadisticamente menos utilizadas las va bajando al disco.

> **Nota**: El administrador o root, es el usuario con mas permisos del **User Space**, no del **Kernel Space**. Ademas, es el unico usuario que puede instalar cosas en el Kernel Space.

Descriptores

Describen a pedazos de memorias, estructura de datos que armo Intel. Tiene varios datos, pero nos interesa el `SEGMENT BASE`.

`SEGMENT BASE`

Indica donde empieza. Cada segmento de memoria tiene que estar descripto por un descriptor de segmentos, dice donde empieza, termina, si es de datos, si es de pila, etc.

`SEGMENT LIMIT`

- **P**: Bit p, indica si esta presente o no esta presente en memoria. Esto se debe a que existe lo que se conoce como memoria virtual, es decir, memoria que esta en disco pues la memoria esta llena.
- **DPL**: Indica el nivel de privilegios del segmento. Hay 4 niveles de privilegios, pero solo necesitamos dos, unos para kernel y otro para user space.

Stack State Segment(TSS)

El procesador de Intel, nos da una herramiento para guardar el contexto de ejecucion, ya no se lo utiliza, pero por mucho tiempo esta estructura de datos de intel se fue utilizada por linux y windows.

Privilegios de E/S

El procesador tiene dos mecanismos:

1. Campo de 2 bits IOPL en los EFLAGS. Que especifica el minimo nivel de privilegios que de debe tener para poder usar instrucciones de E?S
2. Mapa de bits de E/S en TSS.

Instrucciones sensibles:

- IN
- OUT
- INS
- OUTS
- Cli (Clear interrupt)
- Stu (Set Interrupt)

Paginacion

Objetivos de sistemas operativos:

- Proveer recursos a las apps
- Administrar recursos

Problemas:

- Memoria insuficiente
- Fragmentacion de memoria
- Seguridad

Esto se logra con abstraccion e interposicion.

Tiene los mismos objetivos que segmentacion, pero lo hace de una manera diferente. Divide al mapa de memoria(virtual y fisica) en **paginas**. Cada pagina tiene un tamano fijo, genera mapeo de pag-virtuales(paginas) a pag fisicas(marcos) y tiene esquema de permisos.

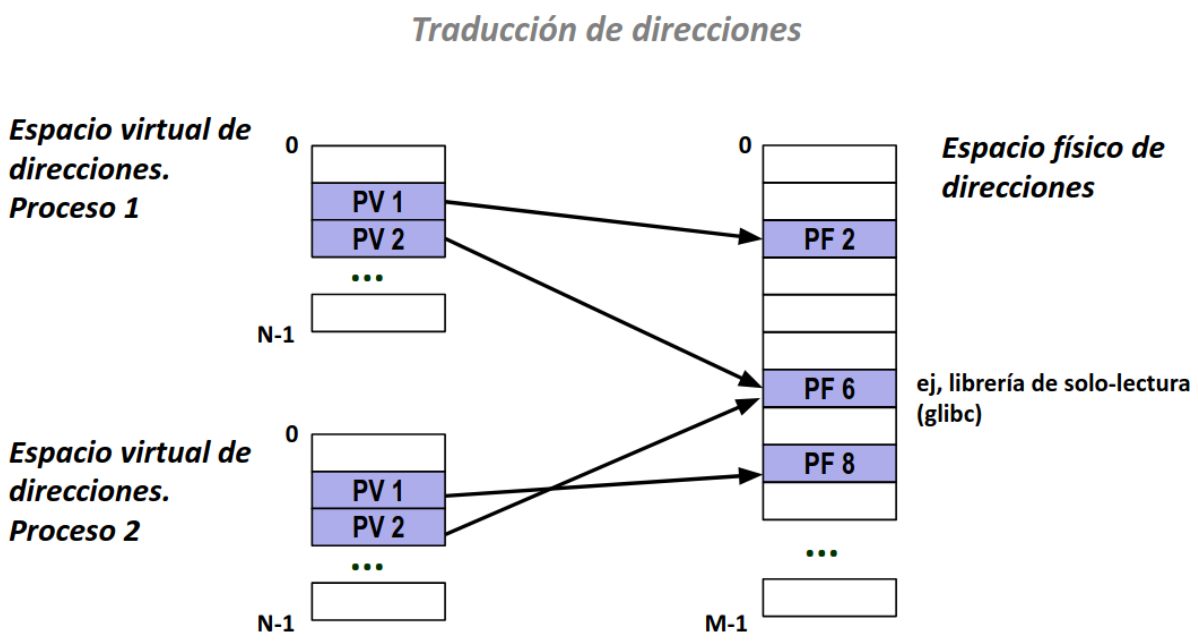


Figure 14: Ejemplo de paginacion con memoria virtual

Ejercicio: Desarrolle un sistema de mapeo para un procesador Intel de 32 bits que puede manejar hasta 4GB de memoria fisica.

1. Que tamano de pagina elige?

Puedo hacer paginas de 4GB, pero tengo solo una, tambien puedo hacer paginas de 2B pero el indice de descriptores va a ser muy grande. Estas no me sirven, Intel en 1985 puso 4KB, para dia de hoy es muy bajo. Vamos por 4KB.

2. En cuantas paginas quedo dividida la memoria fisica? y la virtual?

Hago la cuenta y queda 2^{20} paginas.

3. Como reacciona su sistema para asignar y para liberar memoria?

   Nota: Esto generalmente lo hacen mal, cuando dividis varias paginas de 4KB, cada pagina tiene un offset(del bit 0 al 11) y un numero de pagina(del bit 12 al 31). Pues casa pagina internamente tiene de la direccion 000h a la FFFh.

Cuando hay tantas paginas, surge un problema de tener que indexar a todas. Hay maneras de agruparlos, de manera que si cambio 30 min paginas, solo basta con mencionar el grupo. Entonces partio a la mitad a los bits 12 a 31, donde la primer mitad tiene el indice de directorio y la segunda mitad tiene el indice de pagina(Esto para 32 bits). Sumando los dos indices de obtienen las 2^{20} entradas. Entonces Tomando los 10 bits mas significaticos correspondientes al directorio, se agregan dos 0 al final para tener 3 valores hexa y de ahi se lee. Es virtual, no tengo manera de saber si es la misma memoria fisica, es indireccion.

Luego cada tabla, tiene su P, entonces puede haber un directorio entero en disco o solo una pagina.

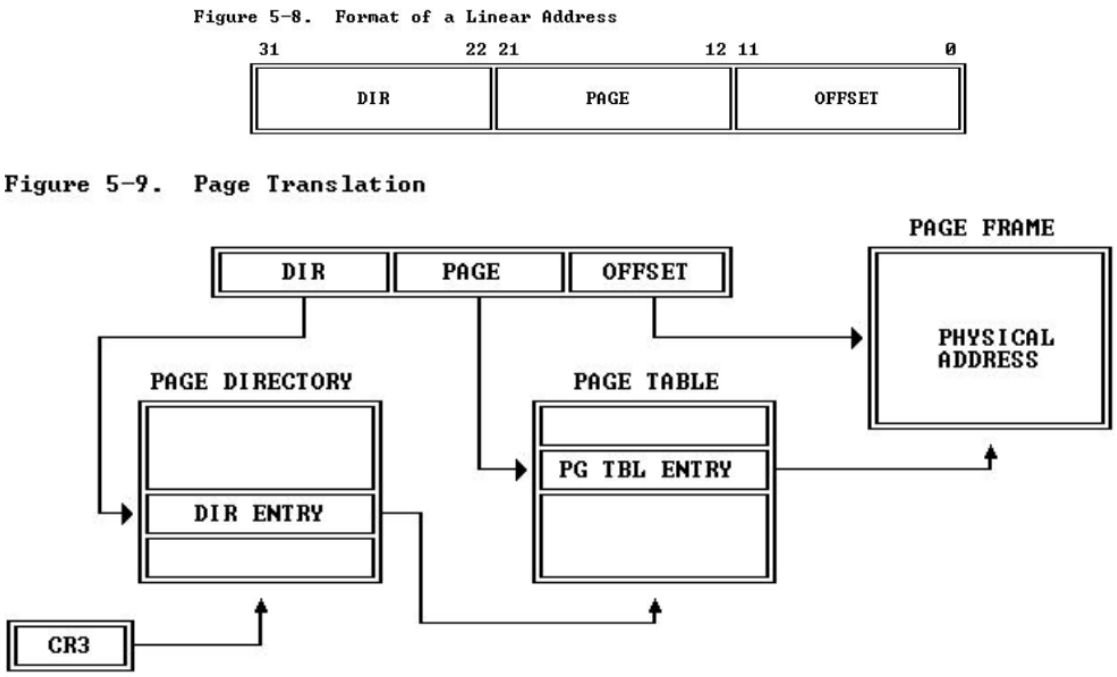


Figure 15: Tabla de paginacion